דוח סיום פרויקט – אבטחת מערכות Web

כותרת הפרויקט:
Exploiting and Mitigating JWT Authentication Vulnerabilities: From Algorithm Confusion to Path Injection
מגישות: שרה שנה זיווי, טליה אבן
מוסד לימודי: מכללת עזריאלי להנדסה, ירושלים

1. תיאור החולשה והרקע התאורטי

מהי החולשה: חולשת JWT Authentication Bypass מתרחשת כאשר שרת מסתמך על תוכן הטוקן ללא אימות קריפטוגרפי של שלמותו. בפרויקט זה הדגמנו שני סוגי כשלים: אימות חתימה לקוי (Algorithm Confusion) וניצול שדות Header (KID Injection).

כיצד היא נוצרת: החולשה נוצרת עקב הסתמכות על פונקציות פענוח (כמו jwt.decode) ללא אכיפת אימות חתימה, או לחילופין, קבלת אלגוריתם none המבטל את הצורך בחתימה. בנוסף, שימוש בפרמטרים מהHeader (כמו kid) לצורך טעינת מפתחות ללא ניקוי קלט מוביל לחולשות הזרקה.

הקשר במערכות ווב: במערכות מודרניות, ה־ JWT משמש כ"כרטיס כניסה" המכיל את זהות המשתמש והרשאותיו. אם השרת לא בודק את ה"חותמת" על הכרטיס, תוקף יכול לשנות את נתוניו באופן עצמאי.

מה התוקף יכול להשיג:
- **Privilege Escalation:** העלאת הרשאות ממשתמש רגיל למנהל מערכת (Admin) .
- **Account Takeover:** התחזות לכל משתמש במערכת על ידי שינוי ה UID ב-Payload
- **Bypass :** מעקף מלא של מנגנון ההזדהות באפליקציה וגישה למידע רגיש.

2. סביבת העבודה

טכנולוגיות:
- **שפת תכנות**: Python
- **Framework**: Flask (ליצירת ה Web API-הפגיע)
- **ספריית קריפטוגרפיה:** PyJWT
- **שרת:** Flask Development Server רץ על Port 5000

כלים להדגמה:
- **Burp Suite / Postman**: לצורך יירוט ושינוי בקשות הHTTP ובדיקת תגובות.
- **jwt.io** : לצורך Debugging ויזואלי של מבנה הטוקן.

3.מימוש והדגמת המתקפה

תיאור ה POC: הקמנו שרת Flask המכיל שני נתיבים פגיעים: /api/jwt-inspect המאפשר כניסה עם אלגוריתם none, וגם- /api/kid-inspect הפגיע להזרקת נתיב במפתח ה-kid.

שלבי הניצול:

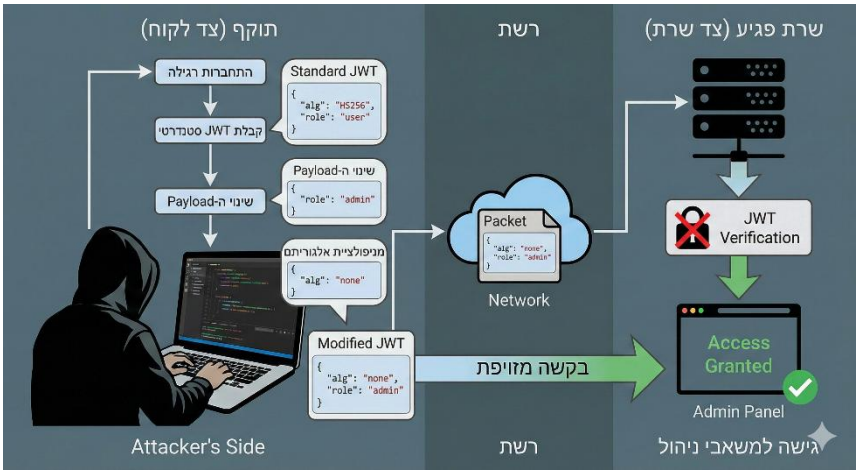
(Scenario 1 - Algorithm Bypass)
1. **זיהוי:** התוקף מתחבר למערכת ומקבל טוקן חתום ב- HS256.
2. **מניפולציה**: שימוש בסקריפט forge_token.py ליצירת טוקן חדש שבו ה Header-מוגדר כ- {"alg": "none"} וה- Payload שונה ל -{"role": "admin"}.
3. **שליחה :**שליחת הטוקן ללא חתימה לשרת

(Scenario 2 - KID Injection)
1. **זיהוי**: השרת משתמש בפרמטר kid כדי לאתר את המפתח בקובץ מקומי.
2. **תקיפה**: שימוש בסקריפט attack_kid.py ליצירת טוקן עם kid המכיל Path Traversal : ..\secret.txt..
3. **ניצול**: השרת קורא את הקובץ secret.txt (המכיל את המפתח הידוע לתוקף) ומשתמש בו כדי לאמת את הטוקן המזויף.

Payloads / קוד / בקשות לדוגמה:
המתקפה התבססה על שליחת טוקן מזויף הכולל את השדה "role": "admin" ב -Payload. בניצול ה- none נעשה שימוש ב- Header מסוג {"alg": "none"} ללא חתימה, ובניצול ה kid- הוזרק הנתיב ..\secret.txt. ל Header-כדי לאלץ את השרת להשתמש במפתח ידוע לתוקף. בקשת ה HTTP-נשלחה דרך Postman עם ה Header-הבא: Authorization: Bearer <forged token>

תוצאה בפועל: השרת מקבל את ה־ Token המזויף כתקין, כלומר מחזיר סטטוס OK 200 עם ההודעה:
"Access Granted! Welcome Admin" ומעניק לתוקף גישה למשאבי ניהול (Admin Panel) למרות שאין לו חתימה תקפה.

דיאגרמה המתארת את שלבי המתקפה:



4. מנגנוני הגנה ומניעה

מדוע ההגנות הקיימות נכשלו:
ההגנה נכשלה כי הקוד ב app.py- ביצע פענוח (Decoding) ללא אימות (Verification) .
המפתח השתמש בפרמטרים שנטרלו את מנגנון האבטחה ויצרו מימוש פגיע:
1. ביטול אימות חתימה: שימוש ב- {"verify_signature": False} בתוך פונקציית ה- decode .
2. אישור אלגוריתם חלש: הכללת "none" ברשימת ה-algorithms המותרים, מה שאפשר לשרת לקבל טוקן ללא חתימה כלל.
3. חוסר בסניטציה ב KID: שימוש ישיר בפרמטר מה- Header (ה- kid) בתוך פונקציית os.path.join מבלי לבדוק אם הוא מכיל תווים של הזרקת נתיב כמו "../"

כיצד ניתן למנוע את החולשה:
כדי למנוע את המעקף, יש להשתמש בפונקציית האימות של הספרייה בצורה "קשיחה". להלן הפונקציות והפרמטרים שחובה ליישם:
א. אכיפת אימות חתימה קריפטוגרפי : במקום רק "לקרוא" את הנתונים, נכריח את השרת לאמת אותם מול המפתח הסודי. אם החתימה לא תתאים (כי התוקף שינה את ה- Payload) הפונקציה תזרוק שגיאה ותעצור את התהליך.
* הפונקציה המאובטחת: jwt.decode(token, SECRET_KEY, algorithms=["HS256"]
*ההבדל הקריטי: הסרת האפשרות ל- 'none' מרשימת האלגוריתמים והסרת הפרמטר- verify_signature: False.
ב. מניעת הזרקת נתיב :(KID Injection) במקום לאפשר לטוקן להחליט מאיזה נתיב לקרוא את המפתח, נשתמש ב"רשימה לבנה" (Whitelist) של מפתחות המוגדרת מראש בקוד.

Best Practices:
- **Whitelist Algorithms :** הגדרת אלגוריתם חתימה יחיד (כמו HS256 או RS256) בשרת, כך שגם אם תוקף ישלח {alg: none} השרת ידחה את הבקשה מיד.
- **Validation Before Use** : לעולם אין להשתמש במידע מה- Payload (כמו- role: admin) לפני שפונקציית ה- decode סיימה בהצלחה את אימות החתימה.
- **Input Sanitization** : התייחסות לכל שדה ב- Header (כמו- kid or jku) כקלט משתמש עוין הדורש סינון מחמיר.
- **Short Expiration:** הגדרת זמן תפוגה (exp) קצר לצמצום חלון התקיפה.
- **Secret Management:** שמירת המפתח הסודי במנהל סודות (Secrets Manager) ולא בקוד המקור.

5. מקורות

- OWASP Top 10: A01:2021 – Broken Access Control
- RFC 7519: JSON Web Token (JWT) standard specification
- PyJWT Documentation: Best practices for secure decoding
- Auth0 Blog: Critical vulnerabilities in JSON Web Token libraries